

Cortex Beyond Snowflake: Integrating AI Across Platforms

Taking the governed assistant to other apps, other AI systems and the tools people already use: REST APIs, Cortex Agents, MCP, Microsoft Teams and Slack

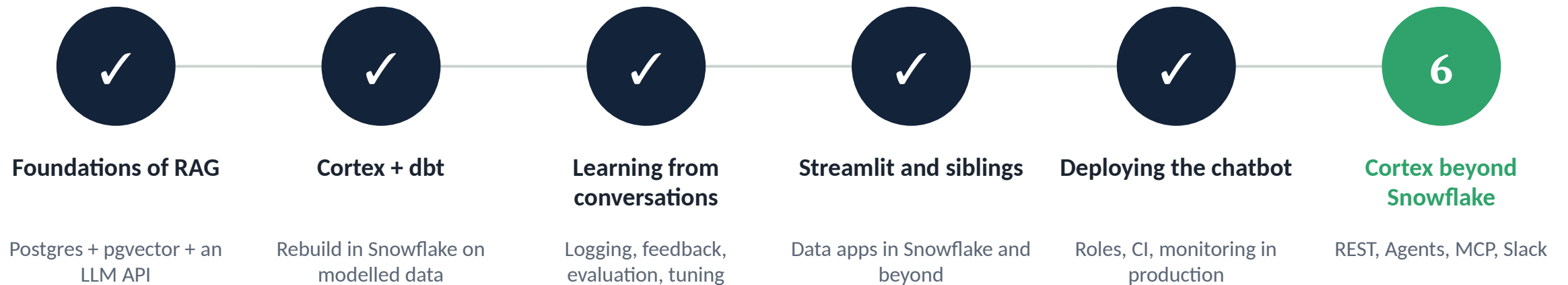


R. Firman Insan Muhammad (Vim)

SnowPro Certified • Data Engineering and AI Trainer

Scenario: Neo Nexus, a loyalty assistant for Caffè Nero. Illustrative only: all data, documents and figures are synthetic.

The same brain, reachable from everywhere



Today's output: a Cortex Agent, a REST client, a Snowflake-managed MCP server, and the assistant answering in Slack or Teams.

By the end of today you can...



Expose

Cortex Search, Analyst, LLMs and agents over REST, with the right authentication for each caller.



Orchestrate

a Cortex Agent that plans across search, analytics and custom tools, replacing the hand-built router.



Connect

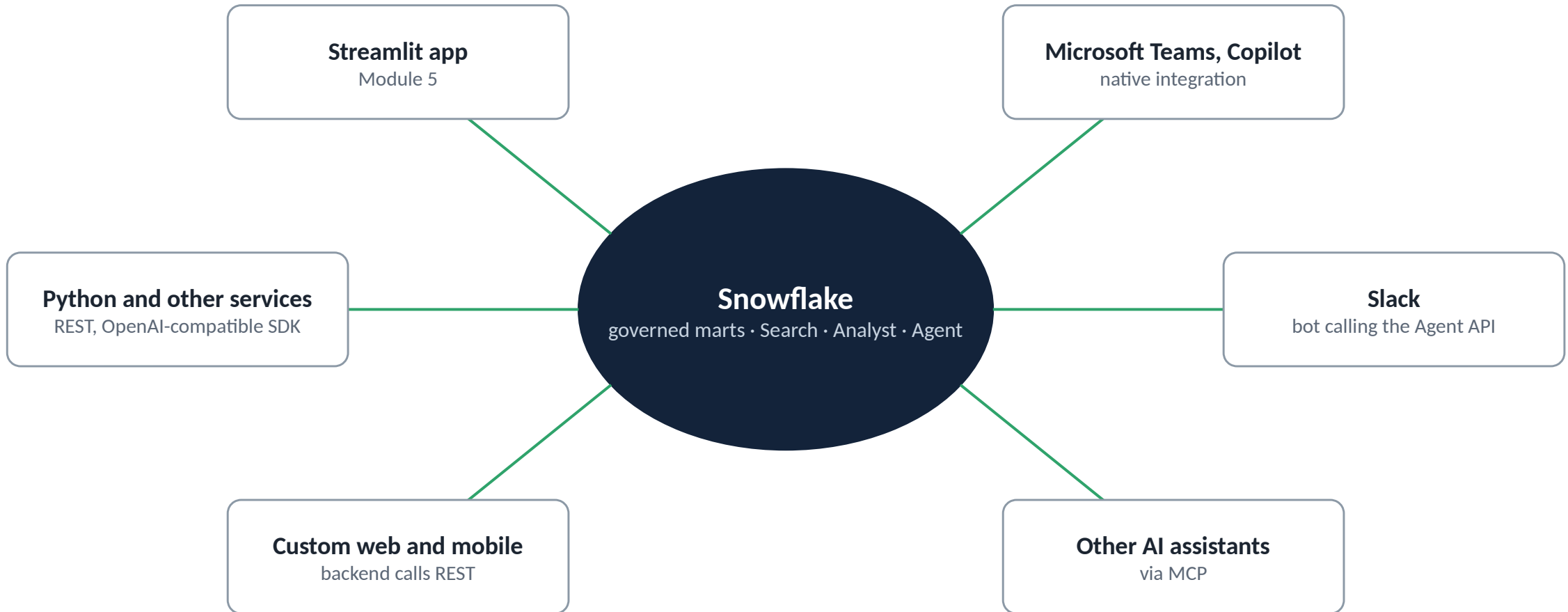
other AI assistants to governed Snowflake data through a Snowflake-managed MCP server.



Govern

every surface, deciding whose identity and permissions each answer runs under.

Move the question to the data, not the data to the question



One governed hub; every surface sends questions in and gets answers out.

Pick the interface by who is calling

Interface	Who calls it	What they get	Typical use
SQL functions	dbt, BI tools, notebooks, any SQL client	AI_COMPLETE, AI_CLASSIFY, SEARCH_PREVIEW, semantic views	Batch enrichment, analytics
REST APIs	Services in any language	Search, Analyst, chat completions, Agents	Custom apps, Slack bots, back ends
MCP server	AI assistants and agent frameworks	Discoverable tools: agent, search, analyst, SQL	Claude, Cursor, other agents
Native integrations	End users	Agents inside Teams and Copilot; Snowflake Intelligence	Business users, no code

Same governed objects behind all four: one search service, one semantic view, one agent.

01

REST APIs

Calling Cortex from any language or platform, and choosing the right way to authenticate

Every capability we built has an HTTP endpoint

Capability	Endpoint (under <code>https://<account>.snowflakecomputing.com</code>)
Chat completions	<code>/api/v2/cortex/v1/chat/completions</code>
Cortex Search	<code>/api/v2/databases/{db}/schemas/{schema}/cortex-search-services/{name}:query</code>
Cortex Analyst	<code>/api/v2/cortex/analyst/message</code>
Cortex Agent (object)	<code>/api/v2/databases/{db}/schemas/{schema}/agents/{name}:run</code>
MCP server	<code>/api/v2/databases/{db}/schemas/{schema}/mcp-servers/{name}</code>

Objects, not endpoints, carry the permissions: grant USAGE on the service or agent, and the API follows.

Choose authentication by whose identity the call carries



Programmatic access token

Simple bearer token for a user. Restrict to a least-privileged role and set an expiry.

Prototypes, MCP clients



OAuth

Carries the end user's identity: their role, row access and masking policies apply.

Acting for a person



Key-pair JWT

A service user signs requests with a private key held in a secrets manager.

Back-end services and jobs

Never in a browser: front ends call your back end; only the back end holds Snowflake credentials.

If it can send HTTPS, it can use the assistant

BASH • CORTEX SEARCH REST

```
curl -X POST \
  "https://$ACCOUNT.snowflakecomputing.com/api/v2/databases/NEO_NEXUS/schemas/SERVING/cortex-search-services/LOYALTY_SEARCH:query" \
  -H "Authorization: Bearer $SNOWFLAKE_PAT" \
  -H "Content-Type: application/json" \
  -d '{"query": "can I redeem a free drink on delivery",
      "columns": ["chunk_id", "chunk_text"],
      "filter": {"@eq": {"channel": "App"}},
      "limit": 5}'
```

Java, Node, Go

same JSON over HTTPS

Workflow tools

UiPath, Power Automate, n8n HTTP steps

Existing LLM code

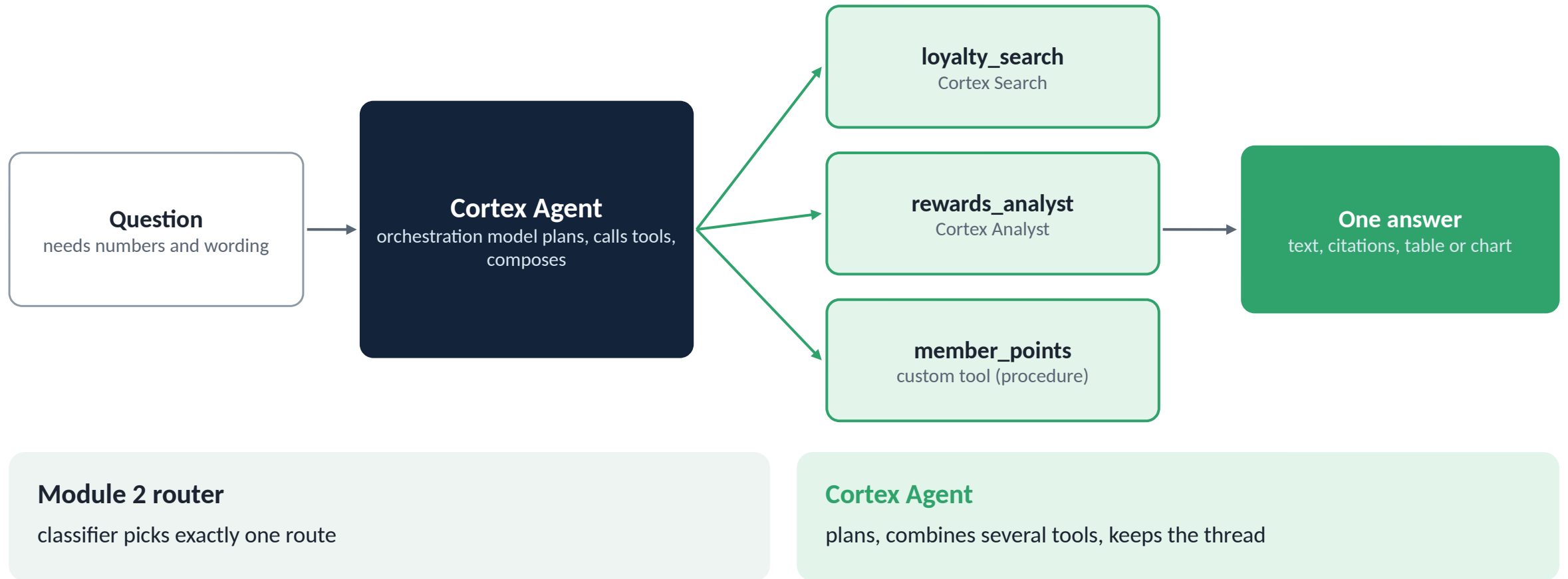
OpenAI SDK with a Snowflake base URL

02

Cortex Agents

From our hand-built router to a managed agent that plans, calls tools and streams its reasoning

An agent plans which tools to use, then answers



The agent is a governed object, defined in YAML

SNOWFLAKE SQL

```
create or replace agent serving.neo_nexus_agent
from specification $$
models:
  orchestration: claude-sonnet-4-5
instructions:
  response: "Answer concisely. Cite programme chunks.
    Say when you are not sure."
  orchestration: "Use loyalty_search for terms and
    FAQs; rewards_analyst for counts and trends."
tools:
  - tool_spec: { type: cortex_search, name: loyalty_search }
  - tool_spec: { type: cortex_analyst_text_to_sql,
    name: rewards_analyst }
tool_resources:
  loyalty_search:
    name: NEO_NEXUS.SERVING.LOYALTY_SEARCH
    max_results: 5
  rewards_analyst:
    semantic_view: NEO_NEXUS.SERVING.REWARDS_SV
    execution_environment:
      { type: warehouse, warehouse: APP_PROD_WH }
$$;
```

Models

which LLM orchestrates

Instructions

how to respond; how to choose tools

Tools

what the agent may use

Tool resources

the real objects behind each tool

One streaming endpoint serves every surface

PYTHON

```
def run_agent(question, token):
    url = (f"https://{ACCOUNT}.snowflakecomputing.com/api/v2"
          "/databases/NEO_NEXUS/schemas/SERVING"
          "/agents/NEO_NEXUS_AGENT:run")
    body = {"messages": [{"role": "user", "content": [
        {"type": "text", "text": question}]}]}
    headers = {"Authorization": f"Bearer {token}",
              "Accept": "text/event-stream"}
    answer, event = [], None
    with requests.post(url, json=body, headers=headers,
                      stream=True) as r:
        for line in r.iter_lines(decode_unicode=True):
            if line.startswith("event:"):
                event = line[6:].strip()
            elif line.startswith("data:") and \
                 event == "response.text.delta":
                answer.append(json.loads(line[5:])["text"])
    return "".join(answer)
```

response.text.delta

answer, as it streams

response.tool_use

which tool, which input

response.table

SQL results to render

response

final aggregated answer

What the agent adds, and what it costs

	Module 2 router	Cortex Agent
Multi-part questions	One route per question	Plans across several tools
Outputs	Text	Text, tables, charts, citations
Conversation state	Your own session state	Threads managed by the API
Extra tools	More code per tool	Declare a tool in the spec
Latency and cost	One classification call	Orchestration on every question
Predictability	Explicit rule, easy to test	Model-planned; test with evals

Decide with the Module 5 gate: run both on the evaluation set and compare quality, latency and cost.

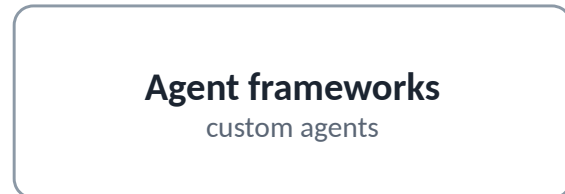
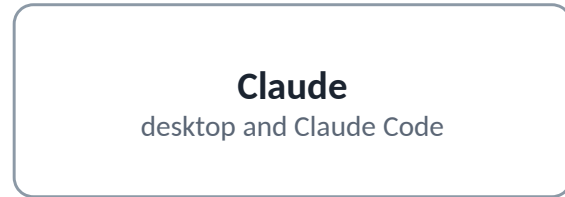
03

MCP

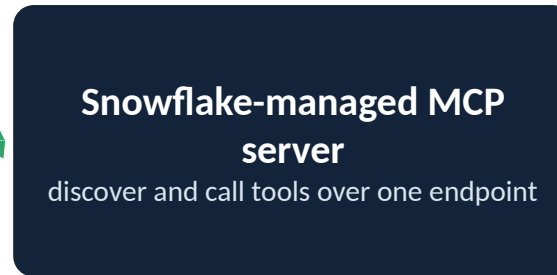
The Model Context Protocol: letting other AI assistants use Snowflake's governed tools

A common plug between AI assistants and tools

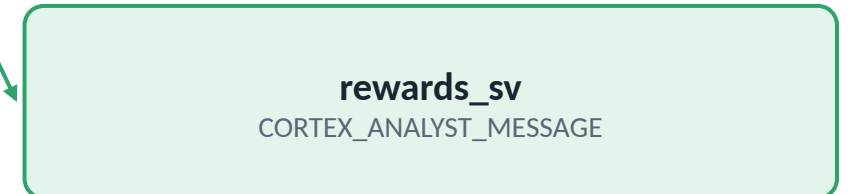
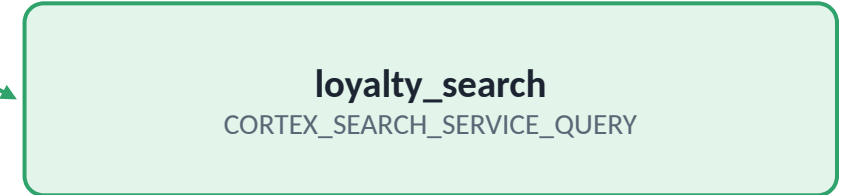
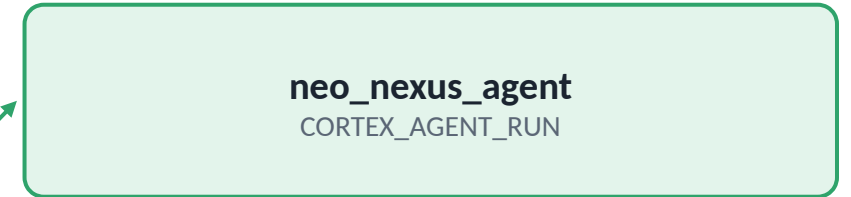
MCP CLIENTS



PROTOCOL



GOVERNED TOOLS



Before MCP: one custom integration per assistant per system. **With MCP:** one server, any compliant client.

One statement publishes governed tools to any MCP client

SNOWFLAKE SQL

```
create or replace mcp server serving.neo_nexus_mcp
from specification $$
tools:
  - name: "neo-nexus-assistant"
    type: "CORTEX_AGENT_RUN"
    identifier: "NEO_NEXUS.SERVING.NEO_NEXUS_AGENT"
    title: "Neo Nexus loyalty assistant"
    description: "Answers questions about Caffè Nero loyalty
      terms and rewards statistics, with citations"
  - name: "loyalty-search"
    type: "CORTEX_SEARCH_SERVICE_QUERY"
    identifier: "NEO_NEXUS.SERVING.LOYALTY_SEARCH"
    title: "Loyalty terms search"
    description: "Searches loyalty terms, FAQs and store guides"
$$;
```

```
https://<account>.snowflakecomputing.com/api/v2/databases/NEO_NEXUS/schemas/SERVING/
mcp-servers/NEO_NEXUS_MCP
```

Tools point at objects

the agent and search service from earlier; nothing new to maintain

Write descriptions for a model

the client's model chooses tools from their descriptions: be specific about what each answers

An MCP server is a door: grant it like one

SNOWFLAKE SQL

```
create role mcp_neo_nexus;  
grant database role snowflake.cortex_agent_user  
  to role mcp_neo_nexus;  
grant usage on warehouse app_prod_wh to role mcp_neo_nexus;  
grant usage on database neo_nexus      to role mcp_neo_nexus;  
grant usage on schema neo_nexus.serving  
  to role mcp_neo_nexus;  
grant usage on mcp server neo_nexus.serving.neo_nexus_mcp  
  to role mcp_neo_nexus;  
grant usage on agent neo_nexus.serving.neo_nexus_agent  
  to role mcp_neo_nexus;  
grant usage on cortex search service  
  neo_nexus.serving.loyalty_search      to role mcp_neo_nexus;
```

OAuth by default

each user's identity and policies flow through

Least-privileged tokens

if you must use a PAT, restrict its role

Read-only SQL, if any

read_only: true and a query timeout

Vet clients and servers

tool text is a prompt-injection surface

04

Where people work

Microsoft Teams and Copilot, Slack, custom front ends, and the identity question every surface raises

Three routes to the people who need answers



Teams and Copilot

Snowflake-provided integration for Cortex Agents. Admin setup; no code to host.

Lowest effort



Slack

A small Bolt bot calls the Agents Run API. Snowflake publishes a quickstart.

Hosting a bot



Custom web and mobile

Your front end calls your back end; only the back end calls the agent.

Full control

All three call the same agent object, under the same grants.

Mention the bot, get a governed answer in the thread

PYTHON • SLACK BOLT

```
import os
from slack_bolt import App
from slack_bolt.adapter.socket_mode import SocketModeHandler

app = App(token=os.environ["SLACK_BOT_TOKEN"])

@app.event("app_mention")
def on_mention(event, say):
    question = event["text"].split(">", 1)[-1].strip()
    thread = event.get("thread_ts", event["ts"])
    say(text="Checking the loyalty library...", thread_ts=thread)
    token = token_for(event["user"])          # whose identity?
    answer = run_agent(question, token)        # Part 2
    log_turn(question, answer, surface="slack")
    say(text=answer, thread_ts=thread)

SocketModeHandler(app, os.environ["SLACK_APP_TOKEN"]).start()
```

Threads

answers stay in a thread, channels stay readable

Logged like any turn

surface = 'slack' in obs.chat_turns

The open question

token_for(user): whose permissions?

Whose permissions does each answer run under?

Pattern	How it works	Policies per person?	Use when
Service identity	one service user for all requests	No	public or non-sensitive data only
Mapped roles	chat groups map to Snowflake roles	Per group	coarse, team-level access is enough
Per-user OAuth	each person signs in; their token is used	Yes	sensitive data, row-level rules, audit

Rule for personal data: if row access policies matter, the person's own identity must reach Snowflake. Anything else is a quiet bypass of your governance.

Cortex plugs into the tools developers already use



Existing LLM code

OpenAI-compatible chat completions: change the key and base URL (Module 2).



Agent frameworks

LangChain and LlamaIndex offer Snowflake integrations for models and retrieval.



BI and notebooks

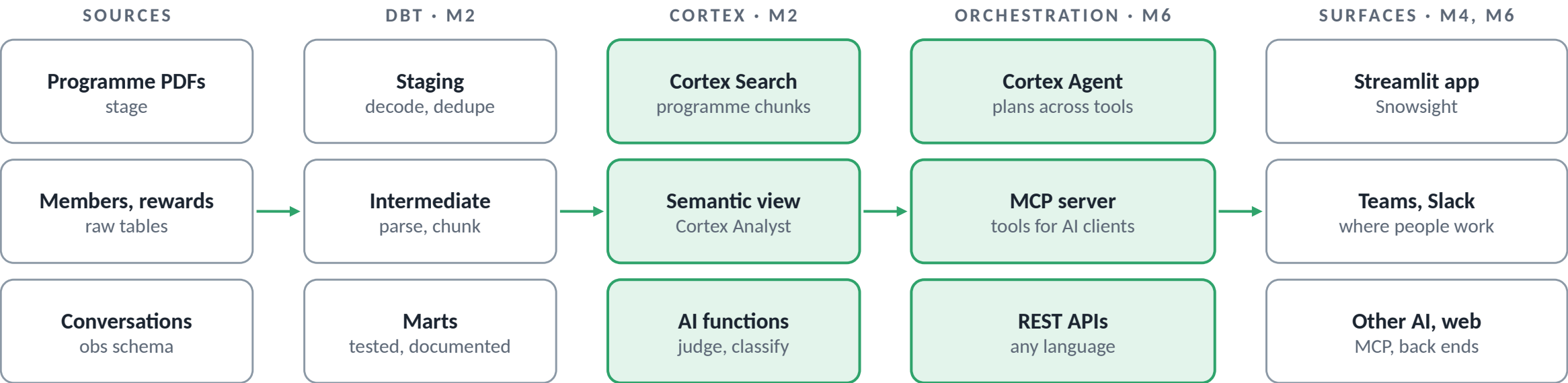
Call AI SQL functions and query semantic views from any SQL client.



Calling out

External access integrations let Snowflake call outside APIs: govern them as egress.

Reference architecture for a governed enterprise assistant



Learn • M3 log every turn • feedback • evaluation set • LLM judge • failure classification • optimisation ladder

Ship • M5 Git • CI • evaluation gate • DEV, TEST, PROD • least-privilege roles • health view, alerts, runbook

Take the assistant beyond Snowflake

- 1 Create the agent**
neo_nexus_agent with search and analyst tools
20 min
- 2 Call it over REST**
run_agent() with streaming; log tool events
15 min
- 3 MCP server**
create, grant, connect an MCP client, ask a question
20 min
- 4 Slack bot**
Bolt bot in a test workspace answering in threads
20 min
- 5 Identity test**
two users, same question: OAuth vs shared token
15 min

Deliverables

- Agent answering a question that needs both search and analyst
- An MCP client using the Snowflake MCP server
- Slack bot answering in a thread, logged to obs
- Identity test result: which pattern respected row access

Stretch: connect the agent to Microsoft Teams with the Snowflake integration

Build a governed assistant for a domain of your choice

Component	Evidence	Weight
Governed data (M2)	dbt project with tests; search service; semantic view	20%
Assistant (M2, M6)	router or agent, justified; cites sources; abstains	15%
Evaluation (M3)	eval set with slices; judge calibration; one measured improvement	20%
App and surfaces (M4, M6)	Streamlit app plus one external surface	15%
Production (M5)	pipeline with gate; roles; health view; rollback	15%
Decision record and demo	ADR on key trade-offs; 10-minute live demo	15%

Suggested weights; adapt to your programme's assessment rules.

Questions worth arguing about

1. A business owner wants the Slack bot live next week on a shared token. What do you say, and what would you accept?
2. What new risks appear when other AI assistants can call your tools through MCP?
3. After six modules, when would you still build the Module 1 DIY stack instead of Cortex?
4. Which single design decision in your capstone would be most expensive to reverse?



References

Snowflake docs: Cortex Agents; Cortex Agents Run API

Snowflake docs: Snowflake-managed MCP server

Snowflake docs: Cortex Agents for Microsoft Teams and Microsoft 365 Copilot

Snowflake docs: Cortex REST API; authentication and programmatic access tokens

Model Context Protocol: specification and documentation, modelcontextprotocol.io

Slack: Bolt for Python documentation

From a vector database to a governed AI platform



The lasting lesson: models change every few months. Governed data, honest evaluation and disciplined delivery are what make an assistant worth trusting.